

아미쿠스렉스(로폼) 프론트엔드 면접 — 완전 준비 문서

- **면접관:** CTO(반드시 참석) + 개발자 2~3명
- **포지션:** 프론트엔드 개발자
- **회사 핵심 축:** Next.js B2B 웹 / 디자인 시스템(Radix UI + Tailwind) / AI 기반 문서 기능
- **면접 안내문 핵심 요구:** ①자기소개·이직사유 필히 ②경력·이력서 내용 질문 ③**향후 업무**("우리 이거 하려는데 할 줄 아나") ④장단점 ⑤지원동기 ⑥비전 기여 ⑦팀 융화(인성) ⑧회사 리서치 필수 ⑨연봉(희망연봉 말하기)

목차

1. 자기소개 (말하기 버전) 2. 이직사유 3. 지원동기 4. 장단점 (업무상 / 개인) 5. 현직 역할·기여 6. 향후 업무 대응 ("이거 할 줄 아나") 7. 회사 비전 기여 8. 인성·팀 융화 9. 회사·대표 리서치 10. 예상 기술질문 + 모범답변 11. 미스매치 방어 답변 12. 역 질문 (지원자 → 면접관) 13. 연봉 14. 면접 당일 체크리스트 15. ⚠ 이력서 심화 대응 (파고들 지점 + 행동질문 + 영어)

1. 자기소개 (말하기 버전 · 약 1분 20초)

굵게 표시한 3지점에서 잠깐 쉬면 리듬이 잡힙니다. 숫자는 외우지 말고 "4개 프로젝트를 혼자" "테이블 대부분 표준화" 정도로 기억.

안녕하세요, 프론트엔드 개발자로 지원한 조성민입니다.

저는 지난 2년 4개월 동안 마이아이비라는 기부·모금 플랫폼에서 **B2C 웹앱, B2B 파트너, 내부 어드민, 트랜잭션 메일** 까지 네 개 프로젝트의 프론트엔드를 혼자 맡아 운영해 왔습니다. 네 개 레포에 합쳐 2,200커밋 정도를 남기며, 화면 기획부터 구현, 배포 후 에러 모니터링까지 한 사람이 이어서 책임지는 방식으로 일했습니다.

그중 가장 오래 붙잡은 일은 **디자인 시스템을 0에서 1로 세운 것**입니다. 입사했을 때 코드에 hex 색상이 수백 군데 흩어져 있고 어드민 테이블도 제각각이었는데, 약 1년에 걸쳐 버튼·아이콘·테이블·토스트 같은 20여 개 카테고리를 토큰화 하고 Storybook으로 문서화했습니다. 공통 컴포넌트를 새로 만들어 어드민 테이블 26개 중 22개를 표준으로 교체했습니다.

또 하나는 **AI 도구를 실제 개발 워크플로우에 녹여 쓰는 방식**입니다. Claude나 Cursor를 단순 자동완성이 아니라, 기획서 초안을 잡거나 수백 곳에 흩어진 레거시를 일괄 정리할 때 아키텍처 파트너처럼 씁니다. 다만 AI 결과를 그대로 믿지 않고 반드시 교차 검증을 거쳐 결함 리스크를 통제하는 걸 원칙으로 합니다.

아미쿠스렉스가 하시는 **Next.js 기반 B2B 웹, Radix UI와 Tailwind 디자인 시스템, AI 기반 문서 기능** 이 세 가지가 제가 지난 2년간 실제로 해온 일과 거의 그대로 겹친다고 느껴 지원했습니다. 법률 문서는 오타 한 줄이 사용자의 권리·계약에 직결되는데, 제가 '신뢰가 흔들리면 사용자가 바로 떠나는' 기부·결제 도메인에서 일해온 감각과 결이 맞닿아 있다고 생각합니다.

입사하면 먼저 코드베이스를 빠르게 읽고, 디자인 시스템과 컴포넌트에 일관된 토대를 만드는 일부터 성실히 보태겠습니다. 잘 부탁드립니다.

짧은 버전 (30초, 시간 없을 때)

"프론트엔드 조성민입니다. 지난 2년여간 기부 플랫폼에서 4개 프로젝트의 프론트엔드를 혼자 운영하며 디자인 시스템을 0→1로 구축하고 AI를 워크플로우에 녹여 써왔습니다. 아미쿠스렉스의 Next.js·디자인 시스템·AI 문서 세 축이 제 경험과 그대로 겹쳐 지원했습니다."

2. 이직사유 (말하기 버전 · 약 40초)

💡 원칙: **현직 회사 절대 깎아내리지 않기**. "힘들어서"가 아니라 "**넓게 → 깊게**" 성장 프레임. 그 답이 왜 아미쿠스렉스인지까지 연결.

지금 회사에서 네 개 프로젝트의 프론트엔드를 혼자 맡으며, 기획부터 배포·모니터링까지 폭넓은 책임 범위를 빠르게 익혔습니다. 성장하기 좋은 환경이었고 지금도 감사하게 생각합니다.

다만 한 사람이 여러 표면을 동시에 보다 보니, 한 영역을 깊게 파는 시간은 의도적으로 만들어야 했습니다. 이제는 **넓게 익힌 것을 한 영역에 더 집중해 깊이를 만들어보고 싶다**는 판단이 들었습니다.

그 영역이 저한테는 **디자인 시스템과 AI 기반 문서 인터페이스**인데, 아미쿠스렉스가 바로 그 두 가지를 핵심으로 하는 회사라서, 그동안 익힌 일하는 방식을 여기서 더 집중적으로 적용해보고 싶어 지원했습니다.

꼬리질문 대비

- **"연봉 때문 아니냐?"** → "처우도 중요하지만 이직의 1순위 이유는 아닙니다. 넓게 vs 깊게의 방향 선택이 먼저였습니다."
- **"4개나 혼자 했으면 여기서도 힘들지 않겠냐?"** → "혼자 다 해봤기에 오히려 팀으로 한 영역을 깊게 파는 환경의 가치를 압니다. 넓게 커버해본 경험이 팀 안에서 인접 영역을 이해하는 데 도움이 될 겁니다."
- **"이직이 처음이냐 / 오래 다닐 거냐?"** → "방향이 맞는 곳에서 길게 일하고 싶습니다. 그래서 회사가 뭘 하는지, 제 강점이 실제로 쓰일지를 보고 지원했습니다."

3. 지원동기

자소서 [3.지원동기] 논리 그대로 — "신뢰 도메인" 연결이 핵심.

지난 2년간 기부·모금 플랫폼을 담당하며 **신뢰가 흔들리는 순간 사용자가 바로 떠난다**는 걸 매일 코드로 마주했습니다. 후원금이 한 번 잘못 결제되면 사용자는 안 돌아오고, 단체가 부정확한 정보로 노출되면 플랫폼 전체 신뢰가 흔들립니다.

아미쿠스렉스를 지원한 이유는 **"누구에게나 법의 가치가 실현되는 세상"**이라는 로퓴의 비전이 제가 익힌 일하는 방식과 결이 같기 때문입니다. 법률 문서는 한 줄의 오타가 사용자의 권리·자산·계약 관계 전체에 영향을 미치는 영역인데, '신뢰가 무너지면 사용자가 떠나는' 임팩트 도메인과 법률 도메인은 **제품 표면의 안정성과 데이터 정합성이 곧 사용자 신뢰의 근거**라는 점에서 매우 닮았다고 봅니다.

또 로퓴이 추구하는 **"어려운 법률문서를 누구든 쉽고 편리하게 완성"**하는 방향은, 제가 PRD를 직접 쓰며 사용자 동선·고객사 신뢰 요구·웹 표현 세 축을 한 문서에서 설계해온 방식과 직접 연결됩니다.

4. 장단점 (업무상 / 개인)

장점 (핵심 1개 + 근거)

가장 큰 장점은 **모호한 현상에 머물지 않고 근본 원인을 추적해 체계화하는 정밀함과 완결성**입니다. 컴포넌트 배치 오차를 사용처에서 스타일로 임시 봉합하는 대신 SVG의 viewBox를 통일해 근본 해결하는 식으로, 한 번 세운 원칙을 코드베이스 전체에 일관되게 적용합니다. 예외 처리 미비로 자라던 Sentry 누적 에러 23건을 하루 만에 전수 조사해 정리한 것도 같은 성향입니다.

단점 (정직하게 + 보완책)

시스템 전체 구조와 큰 그림을 먼저 조망하려는 성향이라, **새 코드베이스에 들어갈 때 큰 그림이 그려지기 전까지 첫 손이 늦은 편**입니다. 이를 보완하려고 '정돈은 점진적으로, 0→1 신규 셋업은 의도적으로 빠르게' 두 모드를 구분해 일합니다. 실제로 B2B 파트너 프론트엔드를 단기간에 캠페인·프로필·계좌 도메인까지 출시한 건 '이 시점엔 신중함보다 추진력'이라는 판단이었습니다.

💡 단점 답변 팁: 단점 → 그게 왜 생기는지 → **어떻게 보완하는지**까지 반드시 세트로. "단점이 없다"거나 "일을 너무 열심히 한다" 같은 회피성 답변 금지.

업무상 장단점 vs 개인 장단점을 따로 물으면

- 업무상 장점: 위 '정밀함·완결성' + 기획을 직접 쓰고 그 위에서 구현하는 사이클.
- 개인 성향 장점: 말로 앞서기보다 작동하는 시스템·정제된 문장으로 소통. 뉴질랜드 4년 거주로 다른 배경의 사람과 협업하는 데 익숙.
- 개인 단점: 완결성을 추구하다 보니 스스로에게 기준이 높은 편 → 우선순위를 정해 '지금 필요한 완성도'를 구분하려 노력.

5. 현직 역할·기여 (숫자로 말하기 — CTO 앞 신뢰 포인트)

한 줄 요약: 1인 프론트엔드로 4개 프로젝트 단독 운영 + 디자인 시스템 0→1 + 레거시 대규모 정돈.

구체 기여 (묻는 만큼 골라 쓰기)

- **디자인 시스템 0→1**: 20여 개 카테고리 토큰화, Storybook + GitHub Pages 자동배포 CI, 공통 컴포넌트 (DataTable·StatusBadge)로 어드민 테이블 26개 중 22개 표준 교체.
- **레거시 대정돈**: 레거시 admin 190개 파일 단일 commit 이관, 미사용 컴포넌트·정적파일 320여 개 일괄 삭제, 아이콘 100여 개→60개(40%↓), 버튼 FillButton 단일화로 90개 파일 일괄 교체.
- **핵심 PRD 작성·구현**: 단체 상세 리디자인 PRD v2.0 — '신뢰 신호 분산 / CTA 휘발성 / 임팩트 추상' 문제 정의, North Star + 7개 Sub-KPI, 페르소나 3그룹, Mermaid 플로우 + 7화면 프로토타입.
- **엔지니어링**: B2B 파트너 프론트 Next.js 제로 셋업, Apollo v3→v4 마이그레이션, deprecated 쿼리 10건 안전 이관, Sentry 4개 레포 표준 통일·에러 23건 전수 조사, generateMetadata SSR 메타데이터 10개 영역 적용, 모금 만들기 LLM 자동채우기 개선·연동, CKEditor 5 운영·커스터마이징.

6. 향후 업무 대응 — "우리 이거 하려는데 할 줄 아나?" (안내문 최대 강조)

아미쿠스렉스가 앞으로 할 일 = Next.js B2B 웹 + Radix/Tailwind 디자인 시스템 + AI 문서 인터페이스(Chat GLD 등). 이 프레임으로 질문이 온다. "해봤다 → 어떻게 → 여기 어떻게 적용" 3단으로 답한다.

회사가 물을 법한 것	답변 뼈대
"Radix + Tailwind로 디자인 시스템 컴포넌트 만들 수 있나?"	"네. Radix는 headless(동작/접근성)라 Tailwind로 스타일 입히는 구조인데, 제가 한 토큰화가 정확히 '동작과 스타일 분리'라 바로 적용 가능합니다."
"AI 문서생성 인터페이스 만들 수 있나?"	"네. 스트리밍 출력 + 검토·수정·재요청 루프 + 에디터 콘텐츠 모델 주입 3가지가 핵심인데, 모금 만들기 LLM 자동채우기에서 검토 루프를 실제로 다뤘습니다."
"법률문서 에디터 커스터마이징 되나?"	"CKEditor 5를 운영·커스터마이징했습니다. 콘텐츠 모델·플러그인·이미지 업로드 파이프라인을 다뤄봐서 빠르게 진입 가능합니다."
"B2B 웹 0→1 셋업 경험 있나?"	"B2B 파트너 프론트를 Next.js로 제로 베이스 셋업해 가입·인증·캠페인·프로필·정산 도메인까지 출시했습니다."

💡 모르는 걸 물으면: "그건 직접 해본 적 없습니다. 다만 [인접 경험]이 있어서 학습 후 빠르게 진입할 수 있습니다." — 우기지 않기.

7. 회사 비전 기여

"누구에게나 법의 가치가 실현되는 세상" 비전에 프론트엔드가 어떻게 기여하냐는 질문.

법이라는 진입 장벽 높은 도메인에서 사용자가 "이 한 장을 내가 완성할 수 있다"는 자신감을 갖게 하는 게 프론트엔드의 역할이라고 봅니다. 그건 화면을 예쁘게 그리는 걸 넘어, 어디서 막히는지·어디서 신뢰가 흔들리는지를 시스템 전체에서 보고 설계하는 사람만 할 수 있는 일입니다.

입사 첫 1년은 Radix + Tailwind 기반 디자인 시스템과 컴포넌트 라이브러리에 일관된 토대를 만드는 데 집중하고, 그 위에서 Chat GLD 같은 AI 문서생성 인터페이스의 사용자 경험을 함께 설계·구현하는 데 도전하고 싶습니다. 사용자가 법률문서 앞에서 느끼는 막막함을 화면에서 덜어내는 일에 기여하겠습니다.

8. 인성·팀 융화 (안내문 명시 — "팀원들과 어떤 자세로 임할지")

합류 초기 자세를 겸손하게, 협업 스타일을 구체적으로.

새 조직에 들어갈 때 가장 중요한 건 제 주장을 앞세우기보다 팀의 소통 방식과 기존 코드베이스의 문화를 먼저 온전히 흡수하는 것이라 생각합니다. 기존 팀원분들이 쌓은 토대 위에 묵묵히 제 몫을 더하는 게 먼저입니다.

협업할 때 저는 말로 앞서기보다 작동하는 시스템과 정제된 문장으로 소통합니다. 1인 프론트엔드로 디자이너·백엔드·기획·마케팅과 협업하며 서로 언어가 달라 생기는 싱크 오류를 줄이는 데 집중했고, Figma↔코드를 일원화한 단일 진실 문서(DSIGN.md), PRD에 엔지니어링 주석을 다는 방식으로 사전 합의를 만들었습니다. 의견이 갈릴 땐 감정이 아니라 데이터 지표와 퍼널 다이어그램으로 합의를 이끌어냅니다.

꼬리질문 대비

- "팀원과 기술 의견 충돌하면?" → "일단 상대 근거를 끝까지 듣고, 사용자 영향·데이터로 판단 기준을 옮깁니다. 그래도 안 정해지면 되돌리기 쉬운 쪽으로 작게 시도해보고 결과로 정합니다. 제 안이 틀렸으면 빠르게 인정합니다."
- "변호사(비개발자)와 일해야 하는데?" → "오히려 익숙합니다. 기획·마케팅 등 비개발 직군과 협업하며 기술 언어를 상대 언어로 번역하는 걸 해왔습니다. 변호사분들의 도메인 지식을 제품 언어로 옮기는 일에 흥미가 있습니다."

9. 회사·대표 리서치 (안내문 강경 경고 — 반드시 숙지)

아미쿠스렉스 = 법률문서 자동작성 플랫폼 '로폼(LawForm)' 운영사

항목	내용
회사명 뜻	라틴어 Amicus(친구) + Lex(법) = "법의 친구"
대표 제품	로폼(LawForm) — 빅데이터·AI 기반 법률문서 자동작성 ("대한민국 1등" 표방)
핵심 가치	비싼 변호사 비용에서 배제된 사람들도 쉽고 저렴하게(개인 1만원 단위~)
BM	B2B + B2C, 구독제 + 사용량 기반
차별점	25년 경력 변호사들이 설계, 현행 법령·조항·판례 최신 데이터 지속 업데이트
AI 방향	Chat GLD 등 AI 문서생성 인터페이스

대표: 정진숙 (변호사 출신 창업자)

- 변호사로 로펌에서 일하다 "시간당 수십만 원 낼 수 있는 사람만 법률 서비스를 받는 현실"에서 배제된 사람들을 돕고자 창업.
- 창업 초기 = 변호사 2명 + 개발자 1명, 단 3명. 현재도 **변호사 : 개발자 ≈ 반반** → 법률 전문성과 개발이 밀착된 조직 문화.

"우리 회사 뭐 하는지 아세요?" 답변

"법률문서를 누구나 쉽고 저렴하게 완성하게 하는 로폼을 만드는 회사죠. 변호사 비용에서 배제되던 분들도 1만원대로 계약서 한 장을 완성할 수 있게 하는 게 핵심이라고 이해했습니다. 변호사와 개발자가 반반인 조직이라는 점도 인상 깊었습니다."

10. 예상 기술질문 + 모범답변

원칙: ①이력서에 적은 건 "왜"까지 ②모르면 정직하게 "여기까지 알고 이건 학습 필요" ③답변 끝을 회사 업무와 연결.

★ 최우선 3영역 (가장 파고들 확률)

Q. Zustand랑 Jotai를 왜 나눠 썼나요?

상태 성격으로 나눴습니다. 근데 그보다 먼저 한 건 **server state와 client state 분리**입니다. 서버 데이터는 Apollo 캐시가 소유하고, UI 상태만 Zustand(여러 값·액션 모은 전역 도메인 store)/Jotai(atom 단위 원자적 구독)가 갖게 했습니다. 서버 데이터를 전역 store에 직접 넣으면 캐싱·무효화·재요청이 얹혀 동기화 지옥이 됩니다.

Q. Apollo v3→v4 마이그레이션에서 뭘 했나요?

MockedProvider 사양이 바뀌어 목 정의를 전수 수정하고 타입 캐스팅을 정리했습니다. 그 과정에서 백엔드와 협업해 deprecated 쿼리·필드 10건을 신규 스키마로 안전 이관했습니다. 마이그레이션 자체보다 '동작이 안 깨지게 넘기는 것'에 집중했습니다.

Q. 디자인 시스템을 0→1로 어떻게 만들었나요?

top-down으로 새 시스템 받기 전에, 코드에 이미 있는 패턴을 카테고리별로 뽑는 **bottom-up**으로 시작했습니다. hex가 수백 곳 흩어져 실제 사용처부터 토근화하고 20여 개 카테고리를 표준화, Storybook 문서화 + 자동배포 CI를 세팅했습니다.

기본기 (아주 기본도 물어본다고 안내받음)

Q. 리스트 렌더링에 key를 왜 쓰나요?

React가 재렌더 시 어떤 엘리먼트가 바뀌고/추가/삭제됐는지 구분하는 식별자입니다. key가 없거나 index를 쓰면 순서 변경·중간 삽입 때 엉뚱한 DOM을 재사용해 상태가 꼬이고 불필요한 리렌더가 납니다. 그래서 안정적인 고유 id를 씁니다.

Q. useEffect는 언제 실행되고 의존성 배열이 왜 중요한가요?

렌더 커밋(페인트) 이후 실행됩니다. 의존성 배열은 "이 값이 바뀔 때만 재실행" 조건인데, 비우면 마운트 1회, 생략하면 매 렌더 실행됩니다. 빠뜨리면 stale closure 버그, 잘못 넣으면 무한루프가 나서 참조하는 값은 다 넣되 함수는 useCallback으로 안정화합니다.

Q. CSR vs SSR 차이는?

CSR은 빈 HTML+JS로 브라우저에서 그려 초기로딩·SEO에 불리, SSR은 서버에서 HTML 완성해 내려 첫 화면·검색에 유리하지만 서버비용·TTFB 부담이 있습니다. myib에서 도메인별 layout 10개에 generateMetadata로 SSR 메타데이터를 적용했습니다.

Q. 클로저가 뭔가요?

함수가 선언될 때의 렉시컬 스코프를 기억해 바깥 함수 종료 후에도 그 변수에 접근하는 특성입니다. React 혹은 stale state 문제가 이 클로저 때문에 생깁니다.

Next.js

Q. App Router와 Pages Router 차이, 둘 다 써봤나요?

양쪽 다 다뤘습니다. Pages는 getServerSideProps/getStaticProps로 데이터 가져오고 파일=페이지, App Router는 서버 컴포넌트(RSC)가 기본이고 layout·loading·error를 파일 컨벤션으로 중첩, fetch 단위 캐싱을 제어합니다. 메타데이터는 App Router에서 generateMetadata로 처리했습니다.

Q. 서버 컴포넌트(RSC)가 뭐가 좋나요?

서버에서만 실행돼 번들에 JS가 안 실려 클라이언트 번들이 줄고, DB·API를 서버에서 바로 접근합니다. 상태·이벤트·브라우저 API가 필요하면 'use client'로 분리해야 하고, 이 서버/클라이언트 경계 설계가 App Router 핵심입니다.

Q. Next 15/16 실제로 썼다는데 특이점?

Next 16에서 params/searchParams가 Promise로 바뀌어 await로 받아야 합니다. 안 하면 빈 param이 들어와 무한 로딩이 나서 그 부분을 정리한 경험이 있습니다.

상태관리 (추가)

Q. Context는 언제 쓰나요?

자주 안 바뀌고 트리 전체가 공유하는 값(테마·인증 세션)에 씁니다. Context는 값이 바뀌면 하위가 통째로 리렌더돼서, 자주 바뀌는 상태엔 Zustand의 선택적 구독을 씁니다.

Apollo / GraphQL (추가)

Q. Apollo 캐시 정규화가 뭔가요?

`_typename + id`로 각 객체를 정규화해 평평한 캐시에 저장합니다. 한 곳에서 갱신하면 그 객체를 참조하는 모든 화면이 같이 갱신됩니다. id 없는 객체나 페이지네이션은 어긋날 수 있어 `keyFields`나 `merge` 함수로 정책을 정합니다.

Q. GraphQL을 REST 대비 왜 쓰나요? 단점은?

필요한 필드만 골라 받아 오버페칭이 줄고 단일 엔드포인트로 여러 리소스를 한 번에 가져옵니다. 단점은 캐싱이 URL 캐싱보다 복잡하고 N+1·과도한 depth 같은 서버 부담이 있어 depth·complexity 제한이 필요합니다.

디자인 시스템 (회사 핵심)

Q. Radix UI를 써봤나요?

프로덕션 직접 도입은 없습니다. 다만 Radix는 접근성·동작만 주는 **headless 컴포넌트**라 스타일을 Tailwind로 입히는 구조인데, 제 토큰화 작업이 정확히 '동작/접근성과 스타일을 분리'하는 일이라 학습보다 적용이 빠를 겁니다.

Q. Tailwind 클래스가 지저분해지는데 관리는?

디자인 토큰을 config에 semantic하게 정의해 bare hex/임의값을 금지하고, 반복 조합은 primitive 컴포넌트로 캡슐화합니다. myib에선 `fontSize·radius·color`를 토큰으로 강제하고 lint로 거버넌스를 걸었습니다.

Q. 컴포넌트 variant 설계는?

시각 조합(색/크기/상태)을 props로 받되 조합 폭발을 막으려 CVA 방식이나 명확한 variant 축으로 제한합니다. FillButton 단일 표준으로 90개 파일을 통일한 게 예시입니다.

AI 기능 (회사 핵심 — Chat GLD)

Q. AI 문서생성 인터페이스를 프론트에서 어떻게 만드나요?

세 가지를 신경 씁니다. ①**스트리밍 UX** — 응답을 토큰 단위로 흘러 체감 지연 감소, ②**검토·수정·재요청 루프** — AI 출력을 그대로 안 쓰고 편집·재생성 가능한 인터페이스, ③**에디터 주입** — 생성 내용을 리치 텍스트 콘텐츠 모델에 안전하게 넣기. myib 모금 만들기 LLM 자동채우기에서 ②를 실제로 다뤘습니다.

Q. 법률 도메인인데 AI 출력을 어떻게 신뢰하나요?

AI 결과를 무비판 수용하지 않고 교차 검증하는 게 원칙입니다. 법률 문서는 오타 한 줄이 권리·계약에 직결되니 AI 출력은 "초안"으로 두고 변호사·사용자 검토 단계를 UX에 명시적으로 넣어야 합니다. 프론트 입장에선 근거 조항 표시, 수정 이력, 확정 전 검토 게이트 같은 신뢰 장치가 핵심입니다.

에디터 (우대사항)

Q. CKEditor로 뭘 했나요?

CKEditor 5 본문 에디터를 운영·커스터마이징했습니다. 리치 텍스트는 HTML을 직접 다루는 게 아니라 **콘텐츠 모델(문서를 트리로 추상화)**을 통해 조작하고, 플러그인으로 기능을 확장하며 이미지 업로드는 별도 어댑터 파이프라인을 씁니다. 이 구조를 다뤄봐 법률문서 에디터 커스터마이징에 빠르게 진입 가능합니다.

성능·품질

Q. 프론트 성능 최적화 뭘 해봤나요?

번들 다이어트로 미사용 파일 320여 개 정리, 아이콘 100여 개→60개(40%↓)로 빌드 비용 감소. 렌더 관점에서 server/client 컴포넌트 분리와 선택적 구독으로 리렌더 최소화를 씁니다.

Q. 에러는 어떻게 잡나요?

4개 레포 Sentry를 단일 표준으로 통일하고 누적 에러 23건을 전수 조사해 글로벌 예외 처리 미비로 인한 잠재 크래시를 사전 제거했습니다. "나면 잡는다"보다 "관측 체계를 통일해 새는 곳을 없앤다"는 접근입니다.

11. 미스매치 방어 답변

원칙: 부족 인정 → 인접 경험으로 다리 → 학습 의지. **절대 우기지 않기**. CTO·개발자는 아는 척을 바로 잡아냄. "여기까진 알고 이걸 학습 필요"가 오히려 신뢰를 줌.

약점	방어 논리
법률 도메인 첫 경험	"처음입니다. 다만 기부·후원 결제·신원 인증은 한 번의 오류가 신뢰로 직결된다는 점에서 법률 문서와 결이 같습니다. 후원금이 잘못 결제되면 안 돌아오는 것처럼 계약서 한 줄 오류도 되돌리기 어렵습니다."
경력 2년 4개월	"연수는 짧을 수 있지만, 1인으로 4개 프로젝트를 단독 운영하며 2,200커밋 이상을 남긴 실무 밀도는 연차 대비 높다고 자부합니다. 기획부터 배포·모니터링까지 끝을 책임져 봤습니다."
Radix UI 미경험	"프로덕션 도입은 없지만 headless(동작/접근성)+Tailwind(스타일) 구조가 제 토큰화 철학과 같아 적용이 빠를 겁니다."
단위테스트 (Vitest/Jest) 약함	"처음부터 촘촘히 짜본 경험은 부족합니다. 대신 Apollo MockedProvider mock 검증과 Playwright·chrome-devtools MCP를 검증 워크플로우에 도입한 경험이 있어 인프라 적용은 빠르고, 코드 직접 작성은 학습 후 진입하겠습니다."
백엔드/풀스택	"프론트 중심이지만 GraphQL 스키마·Enum을 백엔드와 1:1 선제 조율하고 deprecated 쿼리 10건을 신규 스키마로 이관하는 등 API 경계를 넘나든 경험이 있습니다."
AI 활용 vs 구현 구분	"둘 다입니다. 활용은 Claude·Cursor 워크플로우 내재화, 구현은 모금 만들기 LLM 자동채우기 개선·연동입니다. 회사의 AI 문서 인터페이스와 직접 이어집니다."

12. 역질문 (지원자 → 면접관) — "질문 있으세요?" 대비

반드시 2~3개 준비. 회사·역할에 대한 진지한 관심 신호. 연봉/복지만 묻지 말 것.

제품·기술

- "로폼의 AI 문서생성(Chat GLD 등)에서 프론트엔드가 가장 어려운 지점은 어디인가요? 스트리밍 UX인지, 에디터 연동인지, 검토 루프 설계인지 궁금합니다."
- "현재 디자인 시스템은 어느 단계인가요? 제가 합류하면 0→1로 세우는 쪽인지, 있는 걸 고도화하는 쪽인지 알고 싶습니다."
- "프론트엔드 스택에서 가장 기술 부채가 쌓인 영역은 어디인가요?"

조직·일하는 방식

- "변호사와 개발자가 반반인 조직이라고 들었는데, 법률 전문성과 개발이 실제로 어떻게 협업하나요? 프론트 개발자가 변호사분들과 직접 소통하는 자리도 있나요?"
- "입사 후 첫 3개월 동안 이 사람이 이것만큼은 해줬으면 하는 기대가 있다면 무엇인가요?"
- "팀에서 코드 리뷰나 기술 의사결정은 어떤 방식으로 이뤄지나요?"

13. 연봉 (면접 끝 무렵 질문 예정 — 안내받음)

- 이력서 희망연봉 4,500만원(협의 가능) 그대로 말하기.
- 되물으면: "직전 3,425만원에서 상향을 희망하지만 채우는 협의 가능하고, 이직의 1순위 이유는 아닙니다. 역할과 성장 방향이 더 중요합니다."
- 압박("너무 높다") 시: "회사 기준과 제 역할 범위를 보고 협의할 여지는 충분히 있습니다."

14. 면접 당일 체크리스트

콘텐츠 (반드시 입으로 소리내어 연습)

- ☐ 자기소개 1분 20초 — 3번 이상 소리내어
- ☐ 이직사유 40초 + 꼬리질문 3개
- ☐ 지원동기 (신뢰 도메인 연결)
- ☐ 장단점 (단점은 보완책까지 세트)
- ☐ "우리 회사 뭐 하는지 아세요?" — 로폼/정진숙 대표
- ☐ 역질문 3개 준비
- ☐ 희망연봉 4,500 숫자 확정

태도

- ☐ 현직 회사 절대 비방 금지
- ☐ 모르는 건 "학습 후 가능"으로 정직하게 — 우기지 않기
- ☐ 숫자로 신뢰 주기(4개 프로젝트·2,200커밋·22/26·320여 개)
- ☐ 답변 끝을 회사 업무와 연결하는 습관

실무

- ☐ 이력서·경력기술서·지원서 PDF 다시 정독 (내가 쓴 내용 = 질문 소재)
- ☐ 로폼 사이트(lawform.io) 실제로 한 번 써보기 — "직접 써봤다"는 강력한 신호
- ☐ 회사 위치·시간 확인, 10분 전 도착
- ☐ 복장 단정히

15. ⚠ 이력서 심화 대응 (파고들 지점 + 행동질문 + 영어)

이력서·자소서에 적힌 내용에서 나올 수밖에 없는데 앞 장에 없던 것들. [] 표시는 본인 실제 사실로 채워야 하는 자리.

15-1. 타임라인 공백 대응 (거의 확정 질문 — 먼저 물어볼 수도)

Q. "2023년 2월 졸업하고 2024년 2월 입사면, 그 1년은 뭐 하셨어요?"

⚠ 본인 실제 활동으로 채우기. 아래는 프레임 예시: - (취업준비·포트폴리오였다면) "졸업 후 약 1년은 [개발 직무 취업]을 준비하며 개인 프로젝트로 실전 감각을 쌓는 데] 집중했습니다. 그 시기에 만든 것들이 myib 입사 후 0→1 셋업을 빠르게 해내는 밑바탕이 됐습니다." - (부트캠프·학원이었다면) "[OO 과정]을 수료하며 실무 스택(React·Next)을 집중적으로 다졌습니다. 짧은 시간에 프로덕션 수준으로 끌어올리려고 택한 시간이었습니다." - (아르바이트·프리랜스였다면) "[소규모 개발/외주]를 하며 실제 요구사항을 코드로 옮기는 경험을 쌓았습니다."

💡 핵심: 공백을 "빈 시간"이 아니라 "의도적으로 실력을 쌓은 시간"으로. 그 경험이 지금 강점과 어떻게 이어지는지 한 문장으로 연결. 얼버무리면 오히려 의심 사니 짧고 당당하게.

Q. "대학을 오래(2015~2023) 다니셨네요?"

- 군복무 2년(2017.01~2018.10)이 이미 이력서에 있으니 그걸 먼저 언급. "군복무 2년 포함이고, [휴학하며 어학/일경험을 쌓은 기간]이 있었습니다." - ⚠️ 실제 휴학 사유를 짧게. 방어적으로 길게 설명하지 말 것. "그 시간이 지금 일하는 방식에 도움이 됐다"로 마무리.

15-2. AI 자동채우기 — 구체 스토리 (회사 핵심, 가장 깊게 파고들)

⚠️ 이력서 "모금 만들기 LLM 자동채우기 개선·연동" — 회사가 Chat GLD 만드는 곳이라 반드시 깊이 물음. **본인이 실제 한 범위를 정확히 확인해서 과장 없이 답하기.** 아래는 STAR 뼈대:

Q. "AI 자동채우기, 구체적으로 뭘 하는 기능이고 뭘 하셨어요?"

- **상황(S):** 모금 만들기 화면에서 사용자가 제목·설명 등 긴 콘텐츠를 처음부터 쓰기 어려워하는 문제가 있었습니다. - **과제(T):** LLM이 초안을 생성해주되, 사용자가 검토·수정할 수 있어야 했습니다. - **행동(A):** [내가 실제 한 것 — 예: 입력 필드와 LLM API 연동, 로딩/에러 상태 처리, 생성 결과를 폼에 주입, 재생성 버튼]. 특히 AI 출력을 그대로 확정하지 않고 사용자가 편집·재요청하는 루프를 넣었습니다. - **결과(R):** [체감 개선 — 예: 작성 진입 장벽을 낮춤].

💡 꼬리질문 대비: - "LLM 응답이 느리거나 실패하면?" → "로딩 스켈레톤/스피너로 대기 UX를 주고, 실패 시 재시도 와 수동 입력 fallback을 뒀습니다. [실제 처리 방식 확인]" - "스트리밍은?" → 실제로 했으면 "토큰 단위 스트리밍으로 체감 지연을 줄였다", 안 했으면 "이번엔 완성 응답 방식이었고, 스트리밍은 UX 개선 포인트로 알고 있다"고 정직하게. - "프롬프트는 누가 설계?" → 실제 역할대로. FE에서 컨텍스트를 어떻게 구성해 넘겼는지 말할 수 있으면 강점.

15-3. "190개 파일 단일 커밋" 방어 (시니어가 되물을 수 있음)

Q. "190개 파일을 단일 커밋으로? 리뷰·리버트가 어렵지 않나요?"

"일반 기능 개발이라면 저도 잘게 쪼갭니다. 이걸 레거시 디렉토리를 외부 의존성과 함께 통째로 이관하는 작업이라, 중간 상태가 오히려 빌드가 깨지는 비정상 상태였습니다. 그래서 '이관'이라는 하나의 원자적 단위로 묶는 게 리버트도 오히려 깔끔했습니다. 대신 커밋 메시지에 이관 범위·이유를 상세히 남기고, 로직 변경은 0이고 위치 이동만이라는 걸 명확히 해서 리뷰 부담을 줄였습니다." 💡 핵심: "무지성 대량 커밋"이 아니라 "작업 성격에 맞춰 커밋 단위를 판단했다"는 걸 보여주기.

15-4. "프론트엔드인데 PRD를 왜 이렇게 많이?"

Q. "기획(PRD)을 많이 하셨는데, 개발보다 PM을 하고 싶으신 건 아닌가요?"

"제 정체성은 프론트엔드 개발자입니다. PRD를 직접 쓴 이유는 더 나은 화면을 구현하기 위해서였습니다. 1인 개발 환경에서 기획이 모호한 채로 내려오면 결국 구현 단계에서 되돌아가는 비용이 컸습니다. 그래서 사용자 동선·데이터 규칙을 제가 먼저 정리하면 구현이 훨씬 빠르고 정확해졌습니다. 기획은 목적이 아니라 **좋은 구현을 위한 수단**이고, 팀에 PM이 있으면 그 역할을 존중하며 협업합니다." 💡 회사 특성상(변호사=도메인 기획 강함) "저는 구현에 집중하고 도메인 기획은 변호사분들과 협업하겠다"는 태도가 안전.

15-5. 1인 개발의 품질 보증 (bus factor)

Q. "혼자 개발하셨으면 코드 리뷰는 누가? 품질은 어떻게 담보했나요?"

"리뷰어가 없다는 게 가장 큰 리스크라고 봤습니다. 그래서 세 가지로 보완했습니다. ①**셀프 리뷰** — PR을 올리고 시간을 두고 리뷰어 관점으로 다시 봄, ②**AI 교차 검증** — 구조·타입 치환 결과를 AI로 한 번 더 검토, ③**관측 체계** — Sentry로 런타임에서 새는 걸 잡아 사후에라도 품질을 담보. 다만 사람 리뷰의 가치를 알기에, 팀에 합류하면 리뷰 문화에 적극적으로 참여하고 싶은 것도 이직 이유 중 하나입니다." 💡 약점(혼자)을 이직 동기(팀에서 배우고 싶다)로 자연스럽게 전환.

15-6. 행동 질문 (STAR) — 거의 반드시 나옴, 3종 미리 준비

Q. "가장 어려웠던 기술 문제와 해결 과정은?"

Apollo Client v3→v4 마이그레이션. S: 4개 레포 중 하나에서 v4 전환이 필요했고 테스트가 대량으로 깨짐. T: 동작을 깨지 않고 안전 이관. A: MockedProvider 사양 변경을 전수 파악해 목을 재작성, 타입 캐스팅 이슈 정리, 백엔드와 deprecated 쿼리 10건을 신규 스키마로 이관. R: 런타임 장애 없이 전환 완료, 이후 스키마 정합성도 개선.

Q. "실패했거나 후회하는 경험은?"

⚠️ 실제 경험으로. 프레임: "[초기에 큰 그림 없이 성급히 손댔다가 되돌린 경험] — 이후 '진입 단계엔 신중히, 실행 단계엔 빠르게' 두 모드를 구분하게 됐습니다." (단점 답변과 일관되게) 💡 실패담은 반드시 "그래서 뭘 바꿨다"로 끝내기.

Q. "동료와 의견이 충돌한 경험은?"

디자인 시스템을 bottom-up으로 하자고 했을 때. S: 디자이너는 top-down 새 시스템을 원함. T: 방향 합의. A: 코드 실제 사용처를 카테고리별로 뽑아 데이터로 보여주며 "기존 패턴을 먼저 읽자"고 설득, DESIGN.md로 접점을 만듦. R: 양쪽이 같은 문서를 보며 합의, 커뮤니케이션 비용이 줄었다.

15-7. "왜 개발자 / 왜 프론트엔드인가"

Q. "개발은 어떻게 시작했고, 왜 프론트엔드인가요?"

"소프트웨어학과에서 여러 영역을 접했는데, **사용자가 실제로 만지는 표면을 만드는 일**에 가장 끌렸습니다. 제가 만든 화면에서 사용자가 막힘없이 결정을 내리는 걸 볼 때 가장 큰 보람을 느꼈습니다. 특히 '신뢰가 흔들리면 사용자가 떠나는' 도메인에서 일하며, 프론트엔드가 곧 신뢰의 최전선이라는 점이 이 일을 계속하는 이유입니다." ⚠️ 뉴질랜드 배경을 살리려면: "청소년기 뉴질랜드에서 다른 환경에 적응하며 **사용자 입장에서 낮심을 줄이는 감각**을 자연스럽게 익힌 것 같다"는 식으로 연결 가능.

15-8. 영어 자기소개 (30초 — "영어로 한번?" 대비)

뉴질랜드 4년 + "비즈니스 영어 가능"을 적었으니 요청 가능성 있음. 소리내어 2~3번 연습.

"Hi, I'm Sungmin Cho, a frontend engineer. For the past two and a half years, I've single-handedly run the frontend of four products at a donation platform — a B2C app, a B2B partner site, an internal admin, and transactional emails. My main focus has been building a design system from scratch and integrating AI tools into my daily workflow. I applied to Amicus Lex because your three pillars — Next.js-based B2B web, a Radix and Tailwind design system, and AI-powered document features — line up almost exactly with what I've been doing. Thank you."

15-9. 시그니처 기술 디테일 (원리까지 물어봄)

Q. "SVG viewBox를 통일했다는데, 실제로 뭐가 문제였나요?"

"아이콘마다 원본 viewBox 크기가 제각각(예: 20×20, 32×32)이라, 같은 자리에 놓아도 실제 렌더 크기와 광학 중심이 어긋났습니다. 사용처마다 스타일로 보정하면 그 코드가 계속 늘어나서, **viewBox를 24×24로 통일**해 근본에서 정렬을 맞췄습니다. 임시 봉합 대신 원인을 없앤 사례입니다."

Q. "native alert()가 왜 나쁜가요?"

"①**메인 스레드를 블로킹**해서 그동안 다른 JS·렌더가 멈추고, ②**스타일을 못 입혀** 브랜드/디자인 시스템과 따로 놓고, ③**접근성·모바일 UX가 나뉩니다**. 그래서 58개를 일관된 showToast 패턴으로 바꾸며 안 쓰이던 MessageContext 레이어까지 정리했습니다."

Q. "TypeScript는 어느 정도 쓰나요? 제네릭·유틸리티 타입?"

"제네릭으로 재사용 컴포넌트(예: `DataTable<T>`)의 타입 안전성을 확보하고, `Pick·Omit·Partial` 같은 유틸리티 타입으로 API 응답 타입에서 파생 타입을 만듭니다. Apollo codegen으로 GraphQL 스키마에서 타입을 생성해 프론트·백 규칙을 코드로 강제하는 방식도 씁니다." ⚠ 여기서 막히면 안 됨 — 제네릭·유틸리티 타입·타입 좁히기(narrowing)는 예시 하나씩 말할 수 있게.

이 문서 = 면접 전일 통독용 단일 파일. 자소서/이력서 원문은 같은 폴더의 [지원서_아미쿠스렉스_20260609.md/html/pdf](#) 참고. ⚠ [] 표시 자리는 본인 실제 사실로 채운 뒤 최종 통독할 것.